

Script-Controlled ACL

Restrict Record Access Based on Field Value

Naan Mudhalvan – ServiceNow (Skill Wallet)

Project Report

Team ID: SWTID-2026-9038

Team Leader: K. Kumutha

Team Members: R. Siva Dharshini

S. Vayshavi

M. Jasmin

Platform: ServiceNow Personal Developer

Instance **Date of Submission:** 28 September

2026

Contents

1. Introduction

2. Ideation Phase
3. Requirement Analysis
4. Project Design
5. Project Planning & Scheduling
6. Functional and Performance
Testing
7. Results
8. Advantages &
Disadvantages
9. Conclusion
10. Future Scope
11. Appendix

1. INTRODUCTION

1.1 Project Overview

ServiceNow is a cloud platform used by organizations to manage IT services, workflows and business data. Security in ServiceNow is enforced mainly through **Access Control Lists (ACLs)**. An ACL is a rule attached to a table or a field that decides whether a user may read, write, create or delete data. Every time a user opens a list, a form or a field, ServiceNow evaluates the matching ACLs and allows the action only if all conditions are satisfied.

A normal ACL checks only the user's role. This is not enough when one table contains records that belong to different groups. This project builds a **Script-Controlled ACL** on the custom table **u_institution_details**. The ACL uses an Advanced script (server-side JavaScript) that reads the **value of a field on the record** and compares it with the role of the logged-in user. The script sets *answer = true* to allow access and *answer = false* to deny it. Because of this, users who log in with the roles bb1, bb2, bb3 and bb4 see only the records that are meant for them, even though all of them open the same table.

1.2 Purpose

- To protect sensitive institution records from unauthorized users.
- To implement record-level (row-level) security instead of only table-level security.
- To understand how roles, users, ACLs and scripts work together in ServiceNow.
- To learn testing of security rules using the Impersonate User feature.
- To reduce administrator effort by using one flexible rule instead of many separate tables or roles.

2. IDEATION PHASE

2.1 Problem Statement

Organizations store many types of records in a single table. When only role-based ACLs are used, every user who has the table role can see all the records. This leads to data exposure, privacy issues and wrong updates by users who should not touch some records. Administrators need a method to control access record by record, depending on the data inside the record, without creating many tables or a large number of roles.

- **PS-1** – I am: A ServiceNow administrator; Trying to: Let each user see only the institution records meant for them; But: All users with the table role see every record; Because: The ACL checks only the role, not the record data; Which makes me feel: Worried about data leakage
- **PS-2** – I am: A department user; Trying to: Open the institution list and work on my records; But: I can also see records of other groups; Because: No record-level condition is applied; Which makes me feel: Uncomfortable about privacy

2.2 Empathy Map Canvas

Says: "Why can I see records of other groups?", "Only the right people should open this data.", "Can we control it automatically?"

Thinks: Data security is a top priority. Access must depend on who the user is and what the record contains. Manual role management takes too much time.

Team SWTID-2026-9038 | Page 3

Does: Opens the u_institution_details list, switches between test users, compares what each user sees and reports unwanted access.

Feels: Worried when everything is visible, frustrated by repeated manual work, confident and relieved when access is restricted correctly.

Pains: data leakage, many roles to manage, risk of accidental edits. **Gains:** secure records, one reusable rule, less admin work, trust from users.

2.3 Brainstorming

- **Separate table for every group** – Description: Create one table per group so each has its own ACL; Decision: Rejected: too many tables, hard to maintain
- **Role-based ACL only** – Description: Give read role and allow all records; Decision: Rejected: not record-level
- **Business Rule / Data Policy** – Description: Hide or block data through server logic; Decision: Rejected: not the proper security layer
- **Script-Controlled ACL** – Description: Check the field value of each record and the user role inside the ACL script; Decision: **Selected:** native, secure, flexible

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

- **Login** – User Action: User logs in with a role (bb1–bb4); Experience: Neutral; Opportunity: Role-based access starts
- **Open list** – User Action: Navigates to u_institution_details.list; Experience: Expects to see only own records; Opportunity: ACL filters rows automatically
- **Open record** – User Action: Clicks a record; Experience: Allowed or denied by script; Opportunity: Clear, consistent security
- **Edit / create** – User Action: Tries to modify data; Experience: Only permitted records are editable; Opportunity: Write ACL protects data
- **Logout** – User Action: Ends session; Experience: Feels secure; Opportunity: Actions can be audited

3.2 Solution Requirement

Functional requirements

- Create the custom table u_institution_details with fields such as Name, Location, Type and Access Level.
- Create the roles bb1, bb2, bb3 and bb4 and assign them to test users.
- Create a record-type ACL (operation: read) with the Advanced option enabled.
- The script must return true only when the record's field value

matches the user's role. • Records that fail the check must not appear in lists, reports or forms.

Non-functional requirements

Team SWTID-2026-9038 | Page 4

- Security: no unauthorized user can read restricted records.
- Performance: the script must be short and fast.
- Maintainability: the condition can be changed easily.
- Usability: the normal user interface must not change.

3.3 Data Flow Diagram

- 1 – The user opens the list or form of `u_institution_details`.
- 2 – ServiceNow finds the ACLs that match the table and operation.
- 3 – The role and condition checks of the ACL are evaluated.
- 4 – The ACL script runs: it reads the field value from *current* and checks the role using `gs.hasRole()`.
- 5 – The script returns **true** (allow) or **false** (deny).
- 6 – Allowed records are displayed; denied records are hidden from the user.

3.4 Technology Stack

- **Platform** – ServiceNow Personal Developer Instance (PDI)
- **Security feature** – Access Control (ACL) – Advanced script
- **Language** – JavaScript (server-side)
- **Data** – Custom table `u_institution_details`
- **Users and roles** – Test users with roles `bb1`, `bb2`, `bb3`, `bb4`
- **Testing tool** – Impersonate User

4. PROJECT DESIGN

4.1 Problem Solution Fit

The problem is that users with a broad table role can see records that are not meant for them. A Script-Controlled ACL solves this exactly, because it is evaluated for each record at run time and uses the data of the record itself as the condition. The check runs on the server, so it cannot be bypassed from the browser, and no extra table or application is required.

4.2 Proposed Solution

Table design – `u_institution_details`:

- **Name** – Type: String; Purpose: Name of the institution
- **Location** – Type: String; Purpose: City / district
- **Type** – Type: Choice; Purpose: Category of the institution

- **Access Level** – Type: String; Purpose: Holds the role name (bb1–bb4) allowed to view the record

ACL design:

- **Type** – record
- **Operation** – read

Team SWTID-2026-9038 | Page 5

- **Name** – u_institution_details
- **Advanced** – Enabled (script written in the Script field)
- **Requires role** – bb1, bb2, bb3, bb4

ACL script:

```
answer = false;
// the record is allowed only when its Access Level
// field matches a role that the logged-in user holds
if (gs.hasRole(current.u_access_level))
  answer = true;
```

If a record has Access Level = bb1, only users with the role bb1 can read it. The same logic works for bb2, bb3 and bb4. Admin users can still see everything.

4.3 Solution Architecture

- **User layer** – Users with roles bb1 to bb4 log in to the instance.
- **Security layer** – ACL evaluates the table, operation, role and script condition.
- **Data layer** – u_institution_details stores the institution records.
- **Result layer** – The user gets a filtered list and form.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was completed in five tasks. Steps followed during implementation:

1. Log in to the ServiceNow Personal Developer Instance and open the required application scope.
2. Create the custom table u_institution_details and add the fields.
3. Create the roles bb1, bb2, bb3 and bb4 (User Administration > Roles).
4. Create test users and assign one role to each user.
5. Add sample records with different Access Level values.
6. Open System Security > Access Control (ACL) and create a new ACL (type record, operation read) for the table.
7. Enable Advanced, add the script and save.
8. Impersonate each user and verify the list and form access.

- **Task 1** – Activity: PDI setup, table and fields; Owner: K. Kumutha; Status:

Completed • **Task 2** – Activity: Roles bb1–bb4 and users; Owner: R. Siva Dharshini;
Status: Completed • **Task 3** – Activity: Script-Controlled ACL; Owner: S. Vayshavi;
Status: Completed • **Task 4** – Activity: Testing using impersonation; Owner: M.
Jasmin; Status: Completed • **Task 5** – Activity: Documentation and screenshots;
Owner: Whole team; Status: Completed

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Team SWTID-2026-9038 | Page 6

Testing was done by impersonating each user and checking the u_institution_details list and forms.

- **TC-01** – User / Role: User with bb1; Expected: Only records with Access Level bb1 visible; Actual: As expected; Status: Pass
- **TC-02** – User / Role: User with bb2; Expected: Only records with Access Level bb2 visible; Actual: As expected; Status: Pass
- **TC-03** – User / Role: User with bb3 / bb4; Expected: Only own records visible; Actual: As expected; Status: Pass
- **TC-04** – User / Role: User without any role; Expected: No records, access denied; Actual: As expected; Status: Pass
- **TC-05** – User / Role: Admin; Expected: All records visible; Actual: As expected; Status: Pass
- **TC-06** – User / Role: Change Access Level of a record; Expected: Visibility moves to the new role; Actual: As expected; Status: Pass
- **TC-07** – User / Role: Load time of the list; Expected: No noticeable delay; Actual: Loads normally; Status: Pass

7. RESULTS

7.1 Output Screenshots

The Script-Controlled ACL worked correctly. Each user could see only the records that matched the role, and all other records were hidden. The screenshots below show the configuration and the results.

servicenow					System Administrator
Institution Details [u_institution_details]					
All records					
Number	Name	Location	Type	Access Level	
IN0001	Ama Institute of Tech	Chennai	Engineering	bb1	
IN0002	Bharathi Arts College	Coimbatore	Arts	bb2	
IN0003	Cauvery Polytechnic	Trichy	Polytechnic	bb1	
IN0004	Delta Medical College	Madurai	Medical	bb3	
IN0005	Eastern Law School	Salem	Law	bb4	
IN0006	Global Business School	Chennai	Management	bb2	

1 to 6 of 6 rows

Figure 7.1 Table u_institution_details with records (admin view)

servicenow		System Administrator
Access Control u_institution_details (read)		
Type	record	
Operation	read	
Name	institution Details [u_institution_details]	
Active	True	
Advanced	True	
Requires role	bb1, bb2, bb3, bb4	
Script	<pre> answer = false; // allow only if the record Access Level matches a role of the user if (gs.hasRole(current.u_access_level)) answer = true; </pre>	
Update		

Figure 7.2 ACL configuration with Advanced script

servicenow				
impersonating: bb1 user				
Institution Details [u_institution_details]				
all > Access Level = bb1				
Number	Name	Location	Type	Access Level
INS0001	Anna Institute of Tech	Chennai	Engineering	bb1
INS0003	Caaverly Polytechnic	Tirichy	Polytechnic	bb1

1 to 2 of 2 rows

Figure 7.3 List view for the bb1 user (only bb1 records)

servicenow				
impersonating: bb2 user				
Institution Details [u_institution_details]				
all > Access Level = bb2				
Number	Name	Location	Type	Access Level
INS0003	Bharathi Arts College	Coimbatore	Arts	bb2
INS0006	Global Business School	Chennai	Management	bb2

1 to 2 of 2 rows

Figure 7.4 List view for the bb2 user (only bb2 records)

8. ADVANTAGES & DISADVANTAGES

Advantages

- Record-level, fine-grained security
- No extra tables or many roles needed
- Runs on the server and cannot be bypassed from the browser
- Condition is easy to change later
- Works for lists, forms and reports together

Disadvantages

- A script error can block valid users
- Requires JavaScript knowledge to maintain

- Complex scripts can slow large tables
- Every role must be tested carefully
- Wrong field values in records can hide data by mistake

9. CONCLUSION

The project successfully implemented a Script-Controlled ACL on the u_institution_details table that restricts record access based on a field value. Testing with the roles bb1 to bb4 proved that each user sees only the permitted records while unauthorized users are blocked. Through this project the team gained practical knowledge of ServiceNow roles, users, ACL scripting and security testing, and understood how dynamic access control keeps business data safe.

10. FUTURE SCOPE

- Extend the same logic to write, create and delete operations and to field-level ACLs.
- Move the logic to a Script Include so it can be reused on many tables.
- Add group-based, department-based and location-based conditions.
- Log denied access attempts and show them on a security dashboard.
- Apply the approach to real applications such as HR, finance and

student data. **11. APPENDIX**

- **Source Code** – ACL script given in Section 4.2
- **Dataset Link** – Custom table u_institution_details (records created manually in the instance)
- **GitHub & Project Demo Link**
 – <https://github.com/kumuthakumus18-hub/kumutha>
<https://dev193639.service-now.com/welcome.do>